List Structures

- Other basic data structure (besides atomic propositions we have already seen): list
- *List* is a sequence of any number of elements
- Elements can be atoms, atomic propositions, or other terms (including other lists)

```
[apple, prune, grape, kumquat]
[]          (empty list)
[X | Y]     (head X and tail Y)
```

this_is_a_list([apricot, peach, pear]).

this_is_a_list([apple, apricot, peach, pear]).

In query mode, one of the elements of new_list can be dismantled into head and tail with

```
this_is_a_list([New_List_Head | New_List_Tail]).
```

the following are equivalent:

```
[apricot, peach, pear | []]
[apricot, peach | [pear]]
[apricot | [peach, pear]]
```

```
[a]

[X,Y]

[1,2,3,4]

[a,[1,X],[],[],a,[a]]

[1|[2|[3|[4|[]]]]] /* exactly equal to the list [1,2,3,4] */

[1|[2|[3|[4]]]] /* another way of writing the list [1,2,3,4] */

[1,2|[3,4]] /* this is also allowed and is the same [1,2,3,4] */
```

http://www.inf.ed.ac.uk/teaching/courses/aipp/

```
add_element([E|L], E, L).

?-add_element(L, apple , [apricot, peach]).
L = [apple, apricot, peach]
?-add_element(L, apple , []).
L = [apple]

get_element(E, [E|L]).

?-get_element(E, [apricot, peach]).
E = apricot
?-get_element(E, [apricot]).
E = apricot
```

Append Example

```
append([], List, List).
append([Head | List_1], List_2, [Head | List_3]) :-
            append (List_1, List_2, List_3).
```

trace.

append([bob, jo], [jake, darcie], Family).

(1) 1 Call: append([bob, jo], [jake, darcie], _10)?

(2) 2 Call: append([jo], [jake, darcie], _18)?

(3) 3 Call: append([], [jake, darcie], _25)?

(3) 3 Exit: append([], [jake, darcie], [jake, darcie])

(2) 2 Exit: append([jo], [jake, darcie], [jo, jake, darcie])

(1) 1 Exit: append([bob, jo], [jake, darcie], [bob, jo, jake, darcie])

Family = [bob, jo, jake, darcie]

yes

```
?-append(X, Y, [a, b, c]).
X = []
Y = [a, b, c]
;
X = [a]
Y = [b, c]
;
X = [a, b]
Y = [c]
;
X = [a, b, c]
Y = []
```

More Examples

```
member(Element, [Element | _]).
member(Element, [_ | List]) :-
                        member(Element, List).
```

The underscore character means an anonymous variable—it means we do not care what instantiation it might get from unification

- trace.
- member(a, [b, c, d]).
- (1) 1 Call: member(a, [b, c, d])?
- (2) 2 Call: member(a, [c, d])?
- (3) 3 Call: member(a, [d])?
- (4) 4 Call: member(a, [])?
- (4) 4 Fail: member(a, [])
- (3) 3 Fail: member(a, [d])
- (2) 2 Fail: member(a, [c, d])
- (1) 1 Fail: member(a, [b, c, d])
- no
- member(a, [b, a, c]).
- (1) 1 Call: member(a, [b, a, c])?
- (2) 2 Call: member(a, [a, c])?
- (2) 2 Exit: member(a, [a, c])
- (1) 1 Exit: member(a, [b, a, c])
- yes

More Examples

```
reverse([], []).
reverse([Head | Tail], List) :-
  reverse (Tail, Result),
           append (Result, [Head], List).
```

- trace.
- reverse([a, b, c], Q).
- (1) 1 Call: reverse([a, b, c], _6)?
- (2) 2 Call: reverse([b, c], _65636)?
- (3) 3 Call: reverse([c], _65646)?
- (4) 4 Call: reverse([], _65656)?
- (4) 4 Exit: reverse([], [])
- (5) 4 Call: append([], [c], _65646)?
- (5) 4 Exit: append([], [c], [c])
- (3) 3 Exit: reverse([c], [c])
- (6) 3 Call: append([c], [b], _65636)?
- (7) 4 Call: append([], [b], _25)?
- (7) 4 Exit: append([], [b], [b])
- (6) 3 Exit: append([c], [b], [c, b])
- (2) 2 Exit: reverse([b, c], [c, b])
- (8) 2 Call: append([c, b], [a], _6)?
- (9) 3 Call: append([b], [a], _32)?
- (10) 4 Call: append([], [a], _39)?
- (10) 4 Exit: append([], [a], [a])
- (9) 3 Exit: append([b], [a], [b, a])
- (8) 2 Exit: append([c, b], [a], [c, b, a])
- (1) 1 Exit: reverse([a, b, c], [c, b, a])

Backtracking

bird(tweety).
bird(oscar).
bird(X) :- feathered(X).
feathered(maggie).
large(oscar).
ostrich(X) :- bird(X), large(X).

trace, (*ostrich*(oscar)).

Call:*ostrich*(oscar)
Call:*bird*(oscar)
Exit:*bird*(oscar)
Call:*large*(oscar)
Exit:*large*(oscar)
Exit:*ostrich*(oscar)

trace, (*ostrich*(X)).

Call:*ostrich*(_4470)
Call:*bird*(_4470)
Exit:*bird*(tweety)
Call:*large*(tweety)
Fail:*large*(tweety)
Redo:*bird*(_4470)
Exit:*bird*(oscar)
Call:*large*(oscar)
Exit:*large*(oscar)
Exit:*ostrich*(oscar)
X = oscar

Unwanted Backtracking & Cut

```
bird(tweety).
bird(oscar).
bird(X) :- feathered(X).
feathered(maggie).
large(oscar).
ostrich(X) :- bird(X), large(X).
```

trace, (*ostrich*(tweety)).
Call:*ostrich*(tweety)
Call:*bird*(tweety)
Exit:*bird*(tweety)
Call:*large*(tweety)
Fail:*large*(tweety)
Redo:*bird*(tweety)
Call:*feathered*(tweety)
Fail:*feathered*(tweety)
Fail:*bird*(tweety)
Fail:*ostrich*(tweety)
false

No need to try tweety again

```
bird(tweety).
bird(oscar).
bird(X) :- feathered(X).
feathered(maggie).
large(oscar).
ostrich(X) :- bird(X), !, large(X).
```

trace, (*ostrich*(tweety)).

Call:*ostrich*(tweety)
Call:*bird*(tweety)
Exit:*bird*(tweety)
Call:*large*(tweety)
Fail:*large*(tweety)
Fail:*ostrich*(tweety)
false

Fail : Forcing failure

```
bird(tweety).
bird(willy).
canary(tweety).
swims(willy).
penguin(X) :- canary(X), !, fail.
penguin(X) :- bird(X), swims(X).
```

If something is a canary it is not a penguin

trace, (*penguin*(willy)).
Call:*penguin*(willy)
Call:*canary*(willy)
Fail:*canary*(willy)
Redo:*penguin*(willy)
Call:*bird*(willy)
Exit:*bird*(willy)
Call:*swims*(willy)
Exit:*swims*(willy)
Exit:*penguin*(willy)
*1***true**

trace, (*penguin*(tweety)).

Call:*penguin*(tweety)
Call:*canary*(tweety)
Exit:*canary*(tweety)
Call:fail
Fail:fail
Fail:*penguin*(tweety)
false

Use cut when seeing if a ground atom is satisfied, not for generating satisfying instances

trace, (*penguin*(X)).
Call:*penguin*(_4902)
Call:*canary*(_4902)
Exit:*canary*(tweety)
Call:fail
Fail:fail
Fail:*penguin*(_4902)
false

Rule order

```
penguin(X) :- bird(X), swims(X).
penguin(X) :- canary(X), !, fail.
bird(X) :- canary(X).
bird(willy).
canary(tweety).
swims(willy).
```

bad

trace, (*penguin*(tweety)).
Call:*penguin*(tweety)
Call:*bird*(tweety)
Call:*canary*(tweety)
Exit:*canary*(tweety)
Exit:*bird*(tweety)
Call:*swims*(tweety)
Fail:*swims*(tweety)
Redo:*penguin*(tweety)
Call:*canary*(tweety)
Exit:*canary*(tweety)
Call:fail
Fail:fail
Fail:*penguin*(tweety)

```
penguin(X) :- canary(X), !, fail.
penguin(X) :- bird(X), swims(X).
bird(X) :- canary(X).
bird(willy).
canary(tweety).
swims(willy).
```

good

trace, (*penguin*(tweety)).

Call:*penguin*(tweety)
Call:*canary*(tweety)
Exit:*canary*(tweety)
Call:fail
Fail:fail
Fail:*penguin*(tweety)

Avoid left recursive rules

```
parent(a,b).
parent(b,c).
ancestor(X,Y):-parent(X,Y).
ancestor(X,Y):-parent(X,Z), ancestor(Y,Z).
```

Do not use the following:

ancestor(X,Y):- **ancestor(X,Z), parent(Y,Z).**

ancestor(X,Y):- **ancestor(X,Z), ancestor(Y,Z).**

Deficiencies of Prolog

- Resolution order control
  - In a pure logic programming environment, the order of attempted matches is nondeterministic and all matches would be attempted concurrently
- The closed-world assumption
  - The only knowledge is what is in the database
- The negation problem
  - Anything not stated in the database is assumed to be false
- Intrinsic limitations
  - It is easy to state a sort process in logic, but difficult to actually do—it doesn't know how to sort

Applications of Logic Programming

- Relational database management systems
- Expert systems
- Natural language processing

Summary

- Symbolic logic provides basis for logic programming
- Logic programs should be nonprocedural
- Prolog statements are facts, rules, or goals
- Resolution is the primary activity of a Prolog interpreter
- Although there are a number of drawbacks with the current state of logic programming it has been used in a number of areas